Docker Network

Practical Assignment Report | Network drivers, commands, testing and observations

1. Aim

To understand Docker networking and practise connecting containers using Docker network drivers.

2. Requirements

• Docker Engine or Docker Desktop installed and running.

• A terminal with Docker CLI access.

• Internet access to download the Nginx and BusyBox images, if needed.

3. Docker network concepts

Bridge network

The default network driver for standalone containers on a single Docker host. Containers on the same user-defined bridge network can communicate with each other by container name.

Host network

On supported Linux environments, a container shares the host network namespace and does not receive its own separate container IP. Port publishing is not used in the usual way with host networking.

None network

Disables normal container networking, leaving only loopback inside the container.

Overlay network

Connects services across multiple Docker Engine hosts, typically in a Docker Swarm environment.

4. Practical 1 — Inspect existing networks

```
docker network ls

docker network inspect bridge
```

The first command lists Docker networks. The second displays details of the default bridge network.

5. Practical 2 — Create a user-defined bridge network

```
docker network create app-net

docker network ls

docker run -dit --name app-box --network app-net busybox sh
docker run -dit --name test-box --network app-net busybox sh
```

Both containers are attached to the same user-defined bridge network.

6. Test container-to-container communication

```
docker exec test-box ping -c 3 app-box
```

Expected result: ping replies from **app-box**. User-defined bridge networks provide container-name DNS resolution. If the BusyBox image does not include the ping applet in a particular build, use a suitable diagnostic image or

install the required tool in a test container.

7. Practical 3 — Publish a container port

```
docker run -d --name web-net --network app-net -p 8082:80 nginx

docker ps
```

Open **http://localhost:8082** in a browser. The host's port 8082 is mapped to port 80 inside the Nginx container.

8. Practical 4 — Inspect and disconnect a container

```
docker network inspect app-net

docker network disconnect app-net test-box

docker network connect app-net test-box
```

The inspect command shows attached containers and network details. The disconnect and connect commands detach and reattach a container.

9. Other network drivers — reference commands

Host network (Linux)

```
docker run --rm --network host nginx
```

Use this only when host networking is supported by your Docker environment. Behaviour differs on Docker Desktop compared with native Linux.

None network

```
docker run --rm --network none busybox ip addr
```

This demonstrates a container with no normal external network interface.

Overlay network (Swarm)

```
docker swarm init

docker network create --driver overlay --attachable swarm-net

docker network ls
```

Overlay networking requires Swarm mode. On a single-node lab, this demonstrates creation of an overlay network; multi-host communication requires additional Swarm nodes.

10. Useful inspection and cleanup commands

```
docker network inspect app-net

docker ps -a

docker rm -f app-box test-box web-net
docker network rm app-net
# If you initialized Swarm only for this exercise:
docker swarm leave --force
```

11. Observations / Results

• A user-defined bridge network allows containers on the same network to communicate by name.

• Port publishing makes a container service reachable through a host port.

• Host, none, and overlay drivers serve different networking requirements.

Submission evidence: take screenshots of docker network ls, successful network creation, both containers attached to app-net, the ping test, and the Nginx page at http://localhost:8082. The expected outputs are examples; use your own screenshots to prove execution.

12. Conclusion

Docker networking enables isolated and controlled communication between containers and external systems. The bridge driver is commonly used for single-host applications, host and none drivers provide specialised network behaviour, and overlay networks support multi-host container communication.